

Arbori

1 Obiective

În lucrarea de față vom prezenta operațiile pe arbori binari de căutare. În Prolog, arborii sunt reprezentați prin structuri recursive. Arborele gol îl reprezentăm prin simbolul *nil*.

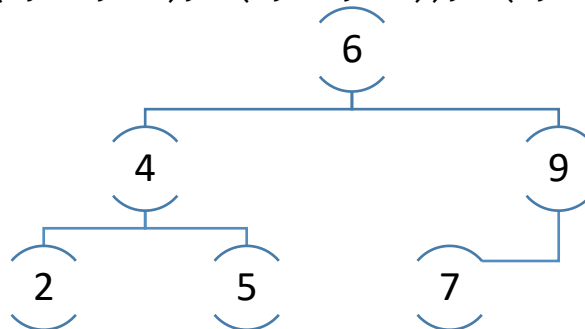
2 Considerații teoretice

2.1 Reprezentare

Arborele binar de căutare este reprezentat printr-o structură cu 3 argumente: cheia nodului curent, subarborele stâng și subarborele drept.

Exemplu:

```
t(6, t(4, t(2,nil,nil), t(5,nil,nil)), t(9, t(7,nil,nil), nil))
```



Pentru a nu scrie de fiecare dată arborele în interpretorul Prolog, puteți „salva” structura în fișier într-un fapt (axiomă).

Exemple:

```
tree1(t(6, t(4,t(2,nil,nil),t(5,nil,nil)), t(9,t(7,nil,nil),nil))).  
tree2(t(8, t(5, nil, t(7, nil, nil)), t(9, nil, t(11, nil, nil)))).
```

...

```
?- tree1(T), operatie_pe_arbore(T,...).
```

2.2 Traversarea arborelui

Există 3 posibilități de a traversa arborele în funcție de ordinea în care sunt procesate nodurile: în-ordine, pre-ordine și post-ordine.

Traversarea în-ordine colectează nodurile din subarborele stâng, apoi nodul curent și în final nodurile din subarborele drept.

```
inorder(t(K,L,R), List):-inorder(L,LL), inorder(R,LR),  
                           append(LL, [K|LR], List).  
inorder(nil, []).
```

```
% cheie, subarbore stâng și subarbore drept
preorder(t(K,L,R), List):-preorder(L,LL), preorder(R, LR),
                           append([K|LL], LR, List).
preorder(nil, []).

% subarbore stâng, subarbore drept și apoi cheia
postorder(t(K,L,R), List):-postorder(L,LL), postorder(R, LR),
                           append(LL, LR,R1), append(R1, [K], List).
postorder(nil, []).
```

Urmărește execuția la:

```
?- tree1(T), inorder(T, L).
?- tree1(T), preorder(T, L).
?- tree1(T), postorder(T, L).
```

2.3 Afișare „pretty”

Ne vom folosi de afișarea „pretty” pentru a vizualiza corectitudinea operațiilor pe arbori. Cea mai ușoară variantă de a afișa un arbore este de a-l traversa în-ordine și fiecare nod să fie afișat la un număr de spații egal cu adâncimea la care se află nodul în arbore. Rădăcina arborelui se află la adâncimea 0.

Exemplu:

```
|           2
|      4
|           5
|6
|           7
|      9
```

```
pretty_print(nil, _).
pretty_print(t(K,L,R), D):-D1 is D+1,
                           pretty_print(L, D1),
                           print_key(K, D),
                           pretty_print(R, D1).
```

```
% predicat care afișează cheia K la D tab-uri față de marginea din
stânga și inserează o linie nouă
print_key(K, D):-D>0, !, D1 is D-1, write('\t'), print_key(K, D1).
print_key(K, _):-write(K), write('\n').
% write('\n') îi echivalent cu predicatul nl
```

Urmărește execuția la:

```
?- tree2(T), pretty_print(T, 0).
```

2.4 Căutare unei chei

Predicatul *search_key(Key, T)* verifică dacă există un nod cu cheia *Key* în arborele *T*. Recurența matematică poate fi formulată astfel:

$$\text{search}(\text{key}, T) = \begin{cases} \text{false} & T = \text{null} \\ \text{true} & T.\text{root}.\text{key} = \text{key} \\ \text{search}(\text{key}, T.\text{left}) & T.\text{root}.\text{key} < \text{key} \\ \text{search}(\text{key}, T.\text{right}) & T.\text{root}.\text{key} > \text{key} \end{cases}$$

În Prolog se va scrie:

```
search_key(Key, t(Key, _, _)):-!.
search_key(Key, t(K, L, _)):-Key<K, !, search_key(Key, L).
search_key(Key, t(_, _, R)):-search_key(Key, R).
```

Urmărește execuția la:

```
?- tree1(T), search_key(5,T).
?- tree1(T), search_key(8,T).
```

2.5 Inserarea unei chei

Primul pas constă în căutarea poziției în arbore unde trebuie inserat nodul cu cheia dată (nodul care se inserează este de tip frunză). Dacă mai există un nod cu aceeași cheie atunci nu se realizează inserarea.

```
insert_key(Key, nil, t(Key, nil, nil)). % inserează cheia în arbore
insert_key(Key, t(Key, L, R), t(Key, L, R)):-!. % cheia există deja
insert_key(Key, t(K,L,R), t(K,NL,R)):- Key<K,!,insert_key(Key,L,NL).
insert_key(Key, t(K,L,R), t(K,L,NR)):- insert_key(Key, R, NR).
```

Urmărește execuția la:

```
?- tree1(T),pretty_print(T,0),insert_key(8,T,T1),pretty_print(T1,0).
?- tree1(T),pretty_print(T,0),insert_key(5,T,T1),pretty_print(T1,0).
?- insert_key(7, nil, T1), insert_key(12, T1, T2), insert_key(6, T2,
T3), insert_key(9, T3, T4), insert_key(3, T4, T5), insert_key(8, T5,
T6), insert_key(3, T6, T7), pretty_print(T7, 0).
```

2.6 Ștergerea unei chei

În cazul ștergerii unei chei trebuie să căutăm nodul care are acea cheie și apoi trebuie să ne asigurăm că în urma ștergerii nodului respectiv, arborele va continua să fie arbore binar de căutare.

```
delete_key(Key, t(Key, L, nil), L):-!.
delete_key(Key, t(Key, nil, R), R):-!.
delete_key(Key, t(Key, L, R), t(Pred,NL,R)):-!,get_pred(L,Pred,NL).
delete_key(Key, t(K,L,R), t(K,NL,R)):-Key<K,!,delete_key(Key,L,NL).
delete_key(Key, t(K,L,R), t(K,L,NR)):-delete_key(Key,R,NR).
```

% caută nodul predecesor

```
get_pred(t(Pred, L, nil), Pred, L):-!.
```

```
get_pred(t(Key, L, R), Pred, t(Key, L, NR)):-get_pred(R, Pred, NR).
```

Urmărește execuția la:

```
?- tree1(T), pretty_print(T, 0), delete_key(5, T, T1),  
pretty_print(T1, 0).
```

```
?- tree1(T), pretty_print(T, 0), delete_key(9, T, T1),  
pretty_print(T1, 0).
```

```
?- tree1(T), pretty_print(T, 0), delete_key(6, T, T1),  
pretty_print(T1, 0).
```

```
?- tree1(T), pretty_print(T, 0), insert_key(8, T, T1),  
pretty_print(T1, 0), delete_key(6, T1, T2), pretty_print(T2, 0),  
insert_key(6, T2, T3), pretty_print(T3, 0).
```

2.7 Înălțime unui arbore

Predicatul $height(T, R)$ calculează distanța maximă între rădăcina arborelui și o frunză. Recurența matematică poate fi formulată astfel:

$$height(T) = \begin{cases} 0 & T = null \\ 1 + \max(height(T.left), height(T.right)) & altfel \end{cases}$$

În Prolog se va scrie:

```
height(nil, 0).
```

```
height(t(_, L, R), H):-height(L, H1),  
                        height(R, H2),  
                        max(H1, H2, H3),  
                        H is H3+1.
```

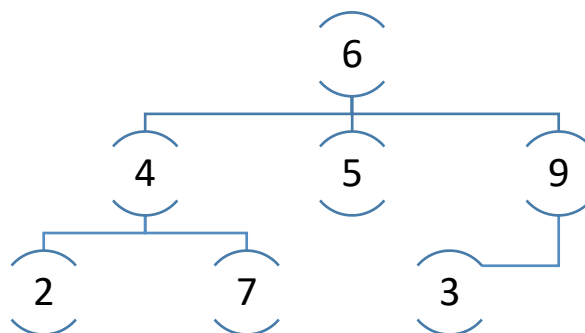
Urmărește execuția la:

```
?- tree1(T), pretty_print(T, 0), height(T, H).
```

```
?- tree1(T), height(T, H), pretty_print(T, 0), insert_key(8, T, T1),  
height(T1, H1), pretty_print(T1, 0).
```

2.8 Arbori ternari

Într-un arbore ternar un nod poate să aibă maxim 3 copii. Arborele îl putem reprezenta folosind o structură de tipul $t(Key, Left, Middle, Right)$.



3 Exerciții

1. Scrieți predicatele care iterează peste elementele unui arbore ternar
 - 1.1. `inorder=Left->Root->Middle->Right`
 - 1.2. `preorder=Root->Left->Middle->Right`
 - 1.3. `postorder=Left->Middle->Right->Root`
2. Scrieți un predicat care calculează înălțimea unui arbore ternar.
3. Rescrieți predicatul `delete_key` folosind nodul succesor.
4. Scrieți un predicat care colectează într-o listă toate cheile din frunzele arborelui binar.
5. Scrieți un predicat care calculează diametrul unui arbore binar

$$diam(T) = \max\{diam(T.left), diam(T.right), height(T.left) + height(T.right) + 1\}$$

6. (*) Scrieți un predicat care colectează într-o listă toate nodurile de la aceeași adâncime din arborele binar.
7. (*) Verificați dacă un arbore binar este simetric. Un arbore binar este simetric dacă subarborele stâng este imaginea în oglindă a subarborelui drept. Ne interesează structura arborelui nu și valorile din noduri.

```
?- tree1(T), symmetric(T).  
false
```

```
?- tree1(T), delete_key(2, T, T1), symmetric(T1).  
T = t(6,t(4,t(2,nil,nil),t(5,nil,nil)),t(9,t(7,nil,nil),nil)),  
T1 = t(6,t(4,nil,t(5,nil,nil)),t(9,t(7,nil,nil),nil)) ? ;  
false
```